

Enhancing Conversational Agents: *Extensions, Evaluation, Reporting*

Project Conversational Agents - Lecture 4

Overview

- **Extending your agent**
- Evaluating your agent
- Report
- Week Summary

Objective

Main goals:

- enhance user experience
- creative, innovative, or sophisticated design ideas
- enhance model performance

Objective

- Improve your grade (check assessment rubric on confluence)
 - Basic Agent
 - Extension and Exclusions
 - Written Report

<https://socialrobotics.atlassian.net/wiki/spaces/PCA21/pages/3117780554/Assessment+Rubric>

Basic Agent - Inclusion (30)

- Intent and Slot Distribution Analysis (3)
- Training Function Implementation (3)
- Robust Intent and Slot Classifier Evaluation Results (7)
- Conversation Patterns and Responses (7)
- Visuals (5)
- Recipe Filtering (5)

Extension and Exclusions (20)

- Exclusion Implementation (4)
- Extensions to Agent Functionality (Recipe Filtering) (4)
- Conversational Competence and Navigation (Extension to Dialogue Patterns) (4)
- Design and User Engagement (Extension to Visual Support) (4)
- Connecting your Pipeline (4)

Many ideas have been suggested already to you

Check out the confluence page on extensions ([PCA 2026](#)):

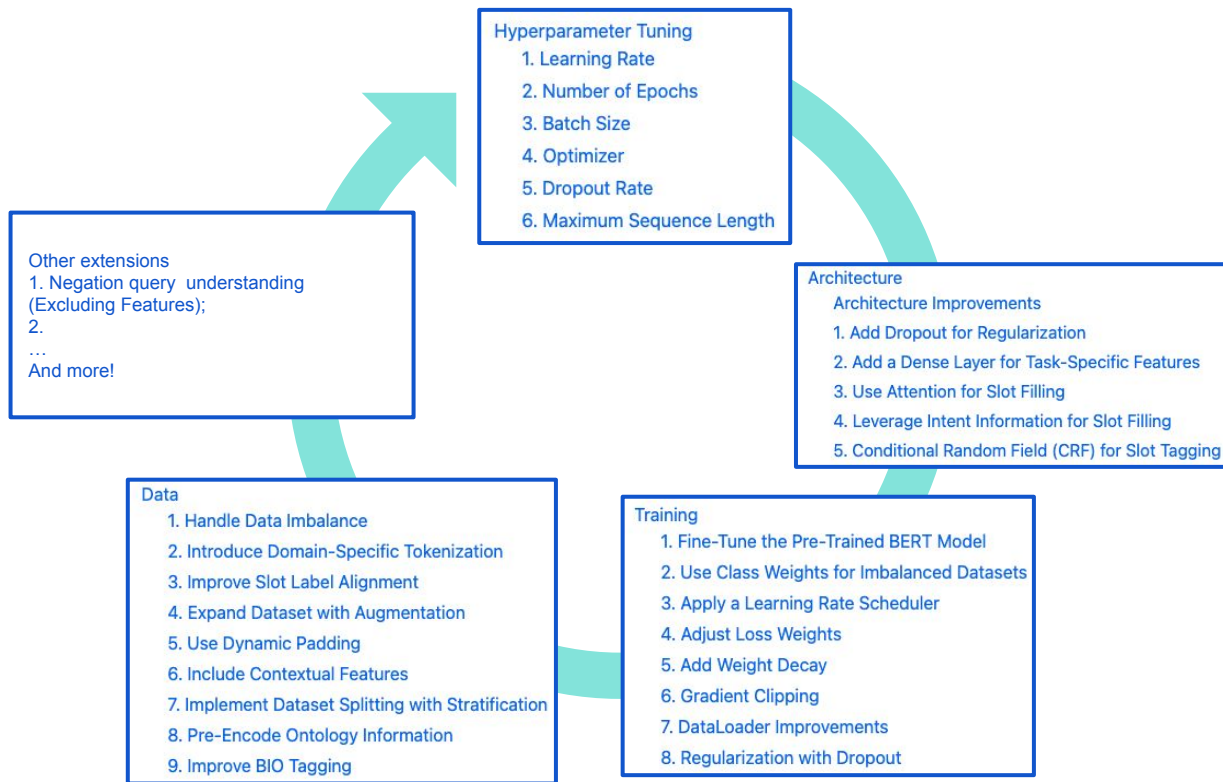
- **Make your agent more conversational and engaging**
 - Refine your agent's responses: ensure variety and contextualize responses (more specific feedback, suggest alternatives, ...)
 - Add introduce small talk capability: The agent could respond to "What's your favorite recipe?" with a lighthearted answer.
 - Add restart conversation capability
 - Enhance personality: Agent personality, style, and characteristics
- **Improve visuals**
 - Visual progress tracker
 - Add a filtering history view
 - Include a rating system to collect user feedback

Many ideas have been suggested already to you

Check out the confluence page on extensions ([PCA 2026](#)):

- **Extend the recipe filtering and selection logic to include new capabilities**
 - Add more recipe filters
 - Check ingredients
 - Additional features
- **Enhance the performance of your intent and slot classifier**
- ...

Enhance your intent and slot classifier

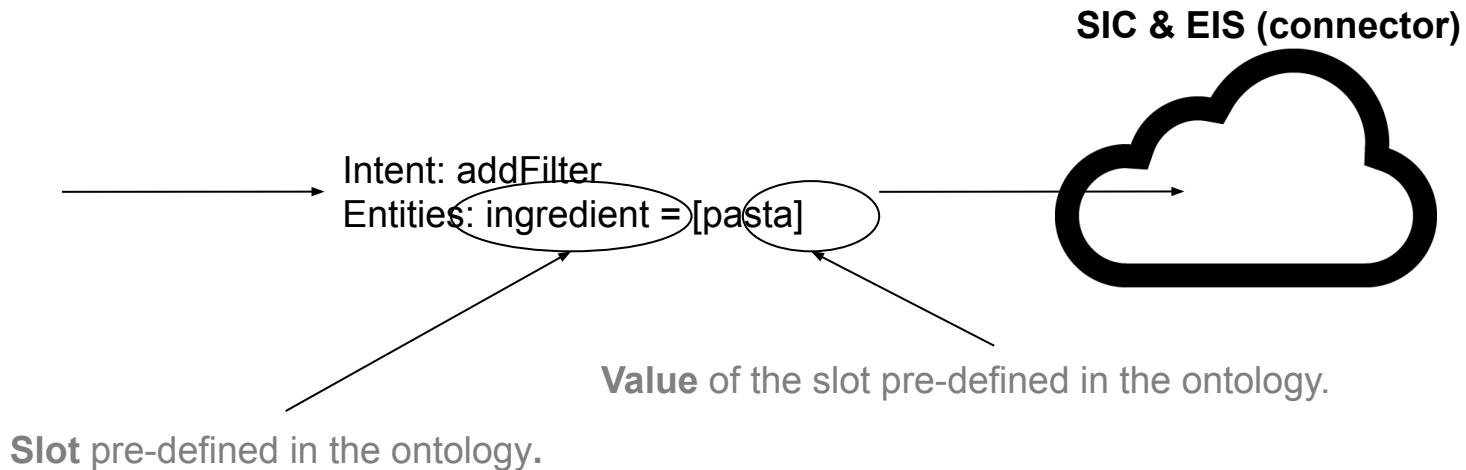


ASR → NLU



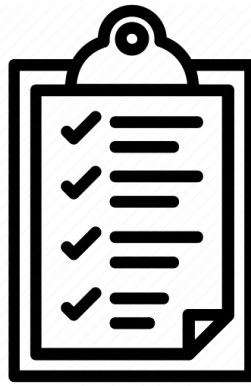
NLU → MARBLE

NLU: BERT



Information from ASR+NLU pipeline to MARBEL

Data Item	Description	Usage in EISComponent
Intent Name	The detected goal of the user's input (e.g., "GetWeather").	Used to trigger actions or send percepts to MARBEL agents.
Entities (Slots)	Key-value pairs extracted from input (e.g., city=London).	Provide specific information for actions or percepts.
Transcript	The raw text entered by the user (spoken or typed).	Sent to the webserver for real-time display.
Confidence Scores	Dialogflow's confidence in the identified intent or slots.	Helps determine whether to trust the NLU result or ask the user for clarification.
Source Type	Indicates whether the request originated from speech or text input.	Adds context to actions or percepts sent to MARBEL.
Recognition Results	Real-time transcription feedback during speech recognition.	Displays transcriptions in real-time on the webserver for UI feedback.



Conversation management

Conversation management actions

Special instructions for handling the agent's agenda and memory; you can find the actions available listed in the `dialog_update.mod2g` file, for example:

- `restart` (reinitialize agenda using the `initialAgenda/1` predicate),
- `terminate` (empty agenda),
- ...

These would be useful, e.g., for implementing a `restart` request (user is stuck and wants to start all over again).

Overview

- Extending your agent
- **Evaluating your agent**
- Report
- Week Summary



Conversational Agent Evaluation

What are quality determiners of (task-based) conversational agents?

PARADISE (PARAdigm for Dialogue System Evaluation)

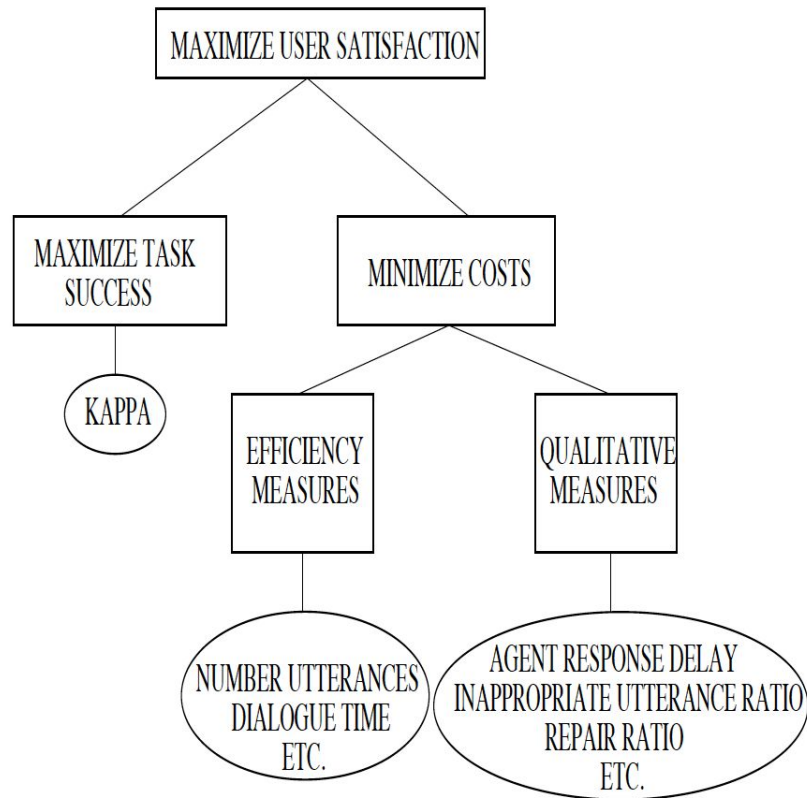


Figure 1: PARADISE's structure of objectives for spoken dialogue performance

User satisfaction

- Make task fun and interesting
- Give conversational cues
- Exude warmth and authenticity
- Connect with user, e.g., read and respond to mood of user

Task success / effectiveness

- Task execution: Test **Happy flow(s)**, e.g., select and confirm recipe, and say goodbye.
 - Breadth of knowledge
 - Ease of use
-
- Correct interpretation
 - Linguistic accuracy
 - Natural interaction

What tasks should your agent be able to perform?

Your agent should be able to complete these tasks as specified in the assignment.

- Recipe selection
 - Recipe instruction
 - Recipe switching
 - Display relevant information
-
- Clarification / help
 - Restart (last topic check)
 - Terminate (say goodbye)

Minimize costs / efficiency

- Turns needed to get the task done (e.g., can you mention multiple feature requests in a single turn?)
- Robust to unexpected input (e.g., handling fallback intents, context mismatches)
- Performing damage control (e.g., how quickly is a user back on track?)

Questions you could ask:

- Is it possible to ask for help during the interaction? (E.g., capability check)
- Ask for clarifications? What if clarification is not available? Is this handled properly?

Interactional efficiency (of a session)

Metric: The theoretical minimum number of turns divided by actual number of turns:

(Some reasonable estimate of the) Happy flow length

Actual number of turns

In our recipe recommendation agent, this means you should count, given a specific session of user and agent, the number of turns that would be minimally needed to complete the opening, recipe recommendation, and closing.

This measure provides an indication of how much ‘extra work’ a user or agent had to do in a conversation.

Interactional efficiency (of a session)

Rule of thumb:

Dialogues during testing with $< .25$ suggests something may need to be improved.

This means more than 4x minimal number of turns are used...

Number of sequences

Metric: The number of sequences that were initiated

I.e. how many patterns were used to initiate a new (sub)sequence?

- How many by the agent?
- How many by the user?

This measure is a measure of **initiative** and gives an indication of user interest because it is a measure of how much a user explored agent capabilities.

Note: Useful to count specific **repair** sequences (e.g. the very generic **b13** pattern) separately from all other sequences, as these rather indicate failure to find something the user was looking for.

How can you test your agent? Navigation testing

What is navigation? *The extent to which a **user** can **control** the dialog flow.*

Examples to design and test for include:

- Basic: Ensure conversation progress, i.e., always at least one user turn is ‘enabled’.
- Permit (easy) reversal of actions.
- Help users recognize, diagnose, and recover from errors.

Navigation testing: Repair scenarios

Test specifically by providing *unexpected* input (utterances/turns). How does your agent handle these?

- **Out-of-context** input → *context mismatch*; test different intents at different points during the conversation
- **Out-of-domain** input → *fallback intent*

How can you test the quality of your agent? Q...

Many ways to test aspects of your agent:

- **Quantitatively:** e.g., how often is a *fallback intent* generated?
- **Qualitatively:** e.g., how sensible/specific are your agent's responses? Illustrate by means of conversation transcripts.

How can you test the quality of your agent? Parts...

Many ways to test aspects of your agent:

- **Components of your system:** ASR/NLU (use e.g., evaluation function in `evaluation.py`, ...), DM (e.g., how often were repair patterns triggered), NLG (variety); either quantitatively or qualitatively.
- **Parts of the conversation:** e.g., related to a pattern at top level; either quantitatively or qualitatively. You can also use **agenda testing** (adapt agenda by removing parts of the conversation).

How can you test the quality of your agent? User...

Many ways to test aspects of your agent:

- **Overall end-user evaluation:** have other people interact with your agent and analyze those interaction sessions (for effectiveness, efficiency, user satisfaction).

Draw lessons learned:

- **what can be improved upon and what are challenges**

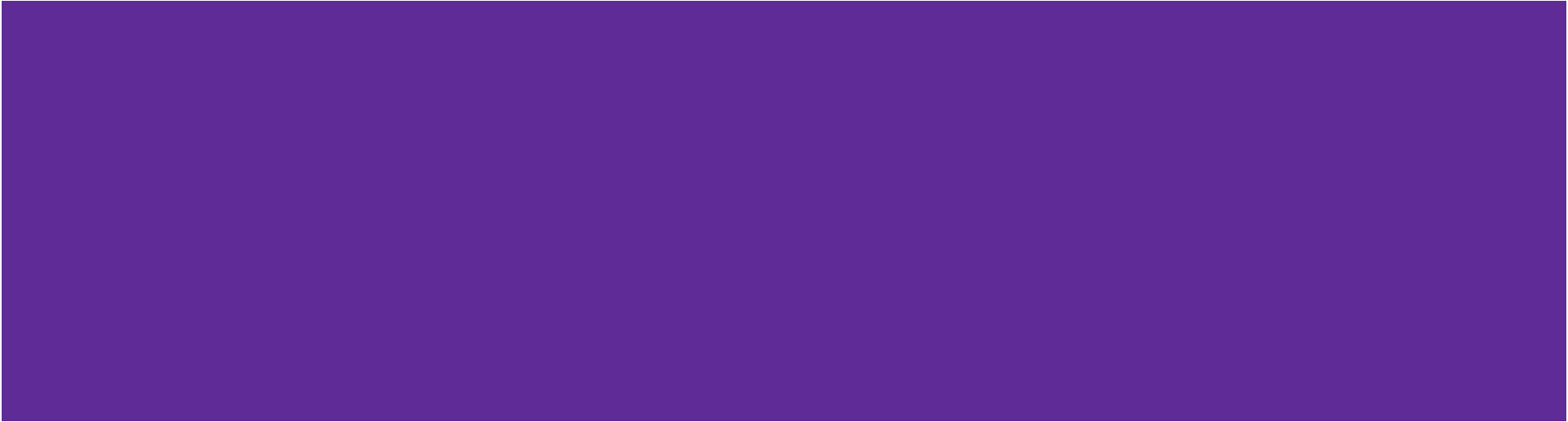
Last but not least? Visual aspects...

- We are aiming for *conversational first* approach
- But cognitive load is an issue during any conversation, when completing a task for sure it also is a challenge to keep track of progress.

Visuals aimed at avoiding confusion:

- Were your users lost during a conversation? Did they get stuck?
- Try to evaluate whether the information presented helped users **navigate** the task and conversation.
- Try to evaluate whether users feel supported by the information that is displayed:
 - Does it give a sense of orientation?
 - Is there any information that the user misses at different moments in the conversation?

Pilot User Study



Why user testing?

Different users converse differently

- Different understanding of the agent
- Asking different questions, making different remarks
- Using different words

Users outside your development team do not know the underlying technology

- You are not the best tester of your own agent!?!
- Black box testing

What is a user study?

- User studies are explorative and discovery-based methods that help you gather insightful and actionable information from your (target) users.

Many different approaches, see e.g. [this website](#) for more info; 2 example approaches:

- **Case studies** are in-depth investigations of a single person, group, event or community. Typically, data are gathered from a variety of sources and by using several different methods
- **Lab experiment:** This type of experiment is conducted in a well-controlled environment – not necessarily a laboratory – and therefore accurate and objective measurements are possible.

How to set up your pilot study? Prepare...

- Invite students from other group for your pilot study (we'll help connect you).
- Think about how to test, e.g.
 - Give concrete task, e.g., 'Find a vegetarian recipe with a cooking time below 15 minutes'
 - Open conversation (user can set his / her own goal)
 - Asking the user during the conversation about responses: "How do you rate this response?"
- Make sure to collect data for each session (logs, observations, ...)

How to set up your pilot study? Instructions...

Give clear guidelines to participants beforehand so they understand what will happen:

- Explain the purpose of your study;
- Explain what participant is expected to do;
- Launch agent (include relaunches);
- Describe what you will do as experimenter: observe, assist when in trouble (that's a decision!), etc.
- Post-interview? Debrief?

How to set up your pilot study: Conducting study

When you conduct the pilot study:

- First, briefly, explain the **procedure** of your user study.
- Ask for **consent** to use the data.
- Make sure each participant has at least **3 conversations** with your agent.
- Collect and record the data for each conversational interaction.
 - (Log files from the MARBEL agent)

How to set up your pilot study? Afterwards...

*“You can find that subjects failed, but they didn’t mind”**

What do users think after conversation with your conversational agent? You can use:

Scales / Questionnaires

Interview user after the conversation with your agent:

- what worked well, what not?
- what did they like, what not so much?

* quoted in Pearl, 2017, Designing Voice User Interfaces, p184.

Analyzing your data

What do you want to find out? What insights/outcomes are you looking for?

What participants did you have? Specific characteristics that are noticeable?

- **Effectiveness:** did a participant complete the task?
- **Efficiency:** how many (conversational) turns were used? Which were “inefficient”?
- **Robustness:** how often did agent (mis)understand user?
- **User satisfaction:** how satisfied was a user with the agent’s performance?

Designing your user study: Outcomes

Efficiency: how many turns were used?

- distinguish ‘efficient’ from ‘inefficient’ steps (dialog moves) from a task perspective.
- Up to you to decide whether e.g. checking agent capability is (in)efficient; do this for all intents in your Dialogflow agent.
- count actual turns (dialog moves) in log file and compute % “inefficient” turns

NB: how will you handle fallback intent?

Designing your user study: Outcomes

Robustness: how often did agent (mis)understand user?

→ define what it means that agent (mis)understands user at pattern/intent level:

- fallback intents are misunderstandings
- if b13 pattern is triggered, do you count that as a misunderstanding?
- check for intents that were incorrectly classified

Use a confusion matrix?

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Designing your user study: Outcomes

User satisfaction: how satisfied was a user with the agent's performance?

- define what it means that a user is satisfied; will be a self-report measure such as a small survey
- satisfied with task completion?
- satisfied with conversational interaction?
- use a simple Likert-scale item: “how satisfied were you with ...? Please rate on a scale of 1-5 from very dissatisfied to very satisfied” or something similar.

Task definition not always clear: Skipping sub-tasks

Specify beforehand in detail what 'task completion' means for each task.

Example: users can specify 0-3 features but does not always need to specify one.

Your design of the recipe selection task might consist of the subtasks:

1. specifying one or more feature(s),
2. naming the recipe,
3. confirmation of the recipe choice by the agent.

But a user might also just skip step 1. So you should test 4 different versions of the task where either 0-3 features are selected.

Overview

- Extending your agent
- Evaluating your agent
- **Report**
- Week Summary

End report

Check out Confluence [here](#) for details:

- Make sure to include **group number, student names, emails** and **student nrs**.
- **Max 10 pages** (a page is about 500 words).
- **Submission:** The final report must be uploaded to the main branch of your GitHub repository. Name the file Report.md.
- **Markdown Format:** Use Markdown (.md format) for your report.

End report: Structure

1. Introduction (max 1 page): Introduce and describe your main aims for the conversational agent.

2. How does your conversational agent work? (max 2 pages): Provide a **functional specification** of your agent (what can it do, how does it flow). Make sure that you give a complete documentation of the features of your agent, **so they can be tested based on your description**.

3. Intent and Slot Classifier (1-2 pages): Explain the importance of these components in identifying user intentions and extracting relevant information (e.g., cuisine type, dietary restrictions). Analyze the performance and improvement.

End report: Structure

4. Exclusion (2 pages): In this section, how exclusion works (e.g., excluding ingredients, cuisines, and mealTypes). Describe the approach your team used to implement Exclusion into your model. Integration of MARBEL, Prolog, Python, and ontology updates.

5. Extensions to the Pipeline (1 page): Summarize the work done related to Extensions. Describe additional enhancements: New functionalities, filters, capabilities added. Improvements in user interaction and experience (e.g., better response generation, conversational adaptability). Explain the motivation and impact of these extensions.

End report: Structure

6. Pilot User Study (1 page): In this section, introduce your Setup, report quantitative and qualitative results from the study and analyze the Key findings and lessons learned and implications for future work.

7. Conclusion (max 1 page): Summarize the main outcomes of the project. Reflect on the process and suggest future improvements or extensions.

Overview

- Extending your agent
- Evaluating your agent
- Report
- **Week Summary**

Week Summary

- Extend your agent
- Create pilot user instructions
- Work on report.
- No more lectures.